

Android中ZXing库的使用讲解

ZXing (Zebra Crossing) 是一个开源的条码/二维码扫描、生成库，支持Android、Java等平台，是Android端处理二维码/条码最常用的库之一。下面从**核心功能**、**集成方式**、**基础使用（生成+扫描）**三个维度，简单讲解Android中ZXing的使用。

一、核心功能

ZXing支持常见的条码/二维码类型：

- 二维码：QR Code、Data Matrix 等；
- 一维码：Code 128、Code 39、EAN-13、UPC-A 等；
- 核心能力：**生成二维码/条码图片**、**扫描解析摄像头中的码**、**解析本地图片中的码**。

二、集成方式（推荐简化版）

官方ZXing的集成步骤较繁琐（需处理摄像头、解码逻辑），社区基于ZXing封装了更易用的版本（如ZXing Android Embedded），优先推荐这种方式：

1. 依赖引入（Module的build.gradle）

Code block

```
1  dependencies {
2      // 核心扫码库（适配AndroidX）
3      implementation 'com.journeyapps:zxing-android-embedded:4.3.0'
4      // 核心解码库（必须）
5      implementation 'com.google.zxing:core:3.5.1'
6  }
7
8  // 若使用Android Gradle Plugin 7.0+，需添加（避免资源冲突）
9  android {
10     buildFeatures {
11         viewBinding true // 可选，根据自己项目是否使用
12     }
13     // 兼容Android 12+的摄像头权限
14     namespace "你的包名"
15 }
```

2. 权限配置（AndroidManifest.xml）

扫码需要摄像头权限，生成二维码无需：

```

1 <!-- 摄像头权限 -->
2 <uses-permission android:name="android.permission.CAMERA" />
3 <!-- 适配Android 6.0+动态权限 -->
4 <uses-feature android:name="android.hardware.camera" />
5 <uses-feature android:name="android.hardware.camera.autofocus"
  android:required="false" />

```

三、基础使用

1. 生成二维码（最简单场景）

无需摄像头，直接生成Bitmap格式的二维码图片：

Code block

```

1 import com.google.zxing.BarcodeFormat;
2 import com.google.zxing.client.j2se.MatrixToImageWriter;
3 import com.google.zxing.client.android.MatrixToBitmapWriter; // 安卓端专用
4 import com.google.zxing.qrcode.QRCodeWriter;
5 import com.google.zxing.common.BitMatrix;
6
7 // 生成二维码的工具方法
8 public Bitmap generateQRCode(String content, int width, int height) {
9     try {
10         // 1. 初始化QRCodeWriter
11         QRCodeWriter qrCodeWriter = new QRCodeWriter();
12         // 2. 生成BitMatrix (二维码矩阵)
13         BitMatrix bitMatrix = qrCodeWriter.encode(
14             content, // 要编码的内容 (如文本、链接)
15             BarcodeFormat.QR_CODE, // 码类型 (QR_CODE为二维码)
16             width, height // 二维码宽高 (建议正方形)
17         );
18         // 3. 转换为Bitmap (安卓端专用方法)
19         return MatrixToBitmapWriter.toBitmap(bitMatrix);
20     } catch (Exception e) {
21         e.printStackTrace();
22         return null;
23     }
24 }
25
26 // 调用示例 (在Activity/Fragment中)
27 Bitmap qrBitmap = generateQRCode("https://www.baidu.com", 300, 300);
28 imageView.setImageBitmap(qrBitmap); // 显示到ImageView

```

2. 扫描二维码/条码（核心场景）

使用 `ZXing Android Embedded` 提供的扫码界面，几行代码即可实现：

步骤1：动态申请摄像头权限（Android 6.0+）

Code block

```
1  private static final int REQUEST_CAMERA_PERMISSION = 100;
2
3  // 检查并申请权限
4  private void checkCameraPermission() {
5      if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
6          != PackageManager.PERMISSION_GRANTED) {
7          ActivityCompat.requestPermissions(this,
8              new String[]{Manifest.permission.CAMERA},
9              REQUEST_CAMERA_PERMISSION);
10     } else {
11         // 权限已授予，启动扫码
12         startScan();
13     }
14 }
15
16 // 权限回调
17 @Override
18 public void onRequestPermissionsResult(int requestCode, @NonNull String[]
19     permissions, @NonNull int[] grantResults) {
20     super.onRequestPermissionsResult(requestCode, permissions, grantResults);
21     if (requestCode == REQUEST_CAMERA_PERMISSION) {
22         if (grantResults.length > 0 && grantResults[0] ==
23             PackageManager.PERMISSION_GRANTED) {
24             startScan();
25         } else {
26             Toast.makeText(this, "需要摄像头权限才能扫码",
27                 Toast.LENGTH_SHORT).show();
28         }
29     }
30 }
```

步骤2：启动扫码界面并处理结果

Code block

```
1  private static final int REQUEST_SCAN = 200;
2
3  // 启动扫码
4  private void startScan() {
5      // 配置扫码选项（可选）
6      IntentIntegrator integrator = new IntentIntegrator(this);
```

```

7     integrator.setDesiredBarcodeFormats(IntentIntegrator.QR_CODE); // 只扫描二维
    码 (可改为ALL_CODE_TYPES)
8     integrator.setPrompt("请对准二维码/条码"); // 扫码提示语
9     integrator.setCameraId(0); // 0=后置摄像头, 1=前置
10    integrator.setBeepEnabled(true); // 扫码成功是否响铃
11    integrator.setBarcodeImageEnabled(true); // 是否保存扫码截图
12    // 启动扫码界面
13    integrator.initiateScan();
14 }
15
16 // 处理扫码结果
17 @Override
18 protected void onActivityResult(int requestCode, int resultCode, Intent data) {
19     super.onActivityResult(requestCode, resultCode, data);
20     // 解析扫码结果
21     IntentResult result = IntentIntegrator.parseActivityResult(requestCode,
    resultCode, data);
22     if (result != null) {
23         if (result.getContents() == null) {
24             Toast.makeText(this, "扫码取消", Toast.LENGTH_SHORT).show();
25         } else {
26             // 扫码成功, result.getContents() 是解析后的内容
27             String scanContent = result.getContents();
28             Toast.makeText(this, "扫码结果: " + scanContent,
    Toast.LENGTH_SHORT).show();
29             // 后续处理 (如跳转链接、解析文本等)
30         }
31     }
32 }

```

四、关键补充

1. **生成一维码**: 只需将 `BarcodeFormat.QR_CODE` 改为对应类型 (如 `BarcodeFormat.CODE_128`、`BarcodeFormat.EAN_13`)，其余逻辑和生成二维码一致；
2. **自定义扫码界面**: 若默认界面不符合需求，可基于ZXing核心库自己实现 (需处理摄像头预览、解码逻辑，复杂度较高)；
3. **解析本地图片中的码**: ZXing也支持从相册图片中解析二维码/条码，核心是将Bitmap转为 `BinaryBitmap` 后解码；
4. **兼容性**: `zxing-android-embedded:4.3.0` 适配Android 5.0+，若需更低版本可选择旧版。

总结

ZXing的核心是**编码（生成）** 和**解码（扫描/解析）**，使用社区封装的 `ZXing Android Embedded` 可大幅简化扫码功能的开发，生成二维码则直接调用核心库的API即可。日常开发中，优先使用封装版实现扫码，自定义需求再基于核心库扩展。

（注：文档部分内容可能由 AI 生成）